

일본어 손글씨 채점 시스템

작동 구조 상세 설명서

KanjiVG 획 데이터 기반 · 전이학습 채점 모델 · 교정 피드백 엔진

프로젝트 폴더: 링고/lingo — 2026년 7월

1. 시스템 개요

이 시스템은 사용자가 애플펜슬·손가락·마우스로 입력한 일본어 글씨(획 좌표 시퀀스)를 0~100점으로 채점하고, 단순 평가를 넘어 어느 획을 어떻게 고쳐야 점수가 최대로 오르는지까지 제시하는 딥러닝 파이프라인이다.

핵심 아이디어는 세 가지다.

구성 요소	설명
합성 학습 데이터	KanjiVG 정답 획에 제어된 왜곡을 가해 "학습자 글씨"를 무한 생성. 왜곡 결과를 기하학적으로 재측정해 점수
전이학습 채점 모델	손글씨 인식 모델(자체 사전학습 또는 Chandra OCR)의 인코더를 가져와 채점 헤드를 파인튜닝. 두 가지 백본
교정 피드백 엔진	모델 예측 + 반사실 분석("이 획을 고치면 +X점") + 그래디언트 방향(점을 어느 쪽으로 옮겨야 점수가 오르는

데이터: kanji/ 폴더의 KanjiVG SVG 11,661자. 각 파일은 유니코드 코드포인트 이름(예: 06f22.svg = 漢)이며, 획마다 순서 번호와 3차 베지어 곡선 경로를 담고 있다.

2. 폴더 및 파일 구조

경로	역할
kanji/	KanjiVG SVG 원본 데이터 (11,661자, 획 순서·베지어 곡선 포함)
scorer/	전체 파이프라인 파이썬 패키지 (아래 3~7장에서 상세 설명)
checkpoints/	학습된 모델 가중치 (recognizer.pt / scorer.pt / chandra_scorer.pt)
runpod_train.sh	RunPod GPU 학습 원커맨드 실행 스크립트
requirements-runpod.txt	RunPod 학습용 파이썬 의존성
README-RUNPOD.md	RunPod에서의 학습 절차 문서
parse_kanji.py / run_parser.py	(구버전) 초기 파싱 실험 파일 — 현재 파이프라인에서는 사용하지 않음

scorer/ 내부 파일	역할
kanjivg.py	SVG 파싱: 베지어 곡선 샘플링 → 획별 (P,2) 좌표, 호길이 재샘플링, 정규화
synth.py	왜곡 엔진 + 라벨 자동 생성 (지터·어파인·필순 반전·획 순서 스왑)
data.py	torch Dataset·배치 변환, 문자셋 로딩 필터
model.py	경량 스트로크 Transformer: PointEncoder / Recognizer / Scorer / 손실함수
train_recognizer.py	1단계: 문자 인식 사전학습 (CPU/GPU)
train_scorer.py	2단계: 인코더 전이 + 채점 헤드 학습 (CPU/GPU)
render.py	스트로크 → 시간 인코딩 3채널 이미지 래스터화 (Chandra 경로용)
chandra_scorer.py	Chandra OCR 비전 타워 전이학습 모델 + 피드백
train_chandra.py	Chandra 전이학습 스크립트 (GPU 전용)
feedback.py	교정 피드백 엔진 (매칭·반사실·그래디언트)
score_api.py	KanjiScorer 고수준 추론 API
server.py + static/	로컬 웹 데모 (캔버스 입력, 표준 라이브러리 HTTP 서버)
evaluate_demo.py	E2E 데모: 합성 글씨 채점 + PNG 시각화

3. 데이터 파이프라인

3.1 SVG 파싱 (kanjivg.py)

각 SVG의 <path id="...-sN" d="..."> 요소를 획 번호 N 순서로 정렬해 읽는다. d 속성의 M/m(이동), C/c(3차 베지어), S/s(연속 베지어), L/l(직선) 명령을 해석해 베지어 세그먼트 리스트로 만들고, 세그먼트마다 14개 점을 샘플링해 이어붙인다.

이후 호 길이 기준 균등 재샘플링으로 획마다 정확히 12점(POINTS_PER_STROKE)을 남긴다. 점 간격이 일정해야 모델이 속도 편향 없이 모양을 학습한다. 마지막으로 글자 전체를 중횡비를 유지한 채 [0.05, 0.95] 정사각 박스에 중앙 배치한다(normalize_strokes). 사용자 입력도 추론 시 동일한 정규화를 거치므로 화면 해상도·글자 크기와 무관해진다.

3.2 합성 왜곡과 라벨 자동 생성 (synth.py)

정답 획 리스트에 심각도 $sev \in [0, 1]$ 에 비례하는 왜곡을 가한다.

왜곡 종류	구현	모사하는 오류
전역 어파인	회전·크기·평행이동 (글자 전체)	글씨가 기울거나 치우침
획별 어파인	획마다 독립적인 회전·크기·이동	개별 획의 위치/기울기 오류
저주파 지터	제어점 4개 노이즈를 선형보간해 더함	손떨림, 모양 뭉개짐
방향 반전	확률적으로 획의 점 순서를 뒤집음	필순 방향 오류 (아래→위 등)
순서 스왑	인접한 두 획의 순서를 교환	획 순서 오류

라벨은 왜곡 파라미터가 아니라 왜곡 결과를 기하학적으로 재측정해 만든다. 획 i 에 대해 위치오차 pos (중심 거리)와 모양오차 $shape$ (중심 정렬 후 점별 평균 거리, 정/역방향 중 최소)를 재고, 품질을 다음처럼 정의한다:

$$q = \exp(-(7.0 \cdot shape + 5.5 \cdot pos)) \times (\blacksquare \times 0.6) \times (\blacksquare \times 0.75)$$

전체 점수 라벨은 획 품질의 평균이다. 여러 왜곡이 겹쳐도 라벨이 일관된다는 것이 이 방식의 장점이며, 모델은 이 라벨을 회귀·분류로 배운다.

4. 모델 A — 경량 스트로크 Transformer (2단계 전이학습)

4.1 입력 표현

획들을 이어붙인 점 시퀀스로 표현한다. 점마다 5차원 특징 $[x, y, dx, dy, \text{획시작 플래그}]$ 를 쓰고, 각 점이 몇 번째 획에 속하는지(stroke_ids)를 따로 전달한다. 12점/획 $\times$ 최대 16획 = 최대 192 토큰.

4.2 PointEncoder (공용 백본)

입력투영(5 $\rightarrow$ 128) + 위치 임베딩 + 획 번호 임베딩을 더한 뒤 Transformer 인코더 ($d=128$, 4헤드, 3층, GELU, pre-norm)에 통과시킨다. 획 번호 임베딩 덕분에 모델이 "몇 번째 획"이라는 필순 문맥을 직접 본다.

4.3 1단계 — 인식 사전학습 (train_recognizer.py)

왜곡된 글씨를 입력으로 "어떤 글자인가"를 분류한다(300~11k 클래스). 이 과제를 풀려면 인코더가 획의 모양·배치·순서를 이해해야 하므로, 손글씨 기하 구조에 대한 일반 특징이 백본에 축적된다. 이것이 채점 과제의 사전학습이 된다.

4.4 2단계 — 채점 전이학습 (train_scorer.py)

Scorer는 동일한 PointEncoder 구조를 갖고, 1단계 체크포인트에서 인코더 가중치를 복사해 온다(load_pretrained_encoder). 학습 시 인코더는 낮은 학습률($5e-5$), 새 헤드는 높은 학습률($2e-4$)로 차등 미세조정한다.

순전파: 사용자 글씨와 정답 템플릿을 각각 인코딩 $\rightarrow$ 획 단위 평균 풀링으로 획 임베딩 생성 $\rightarrow$ 교차어텐션(TransformerDecoder 2층)으로 사용자 획이 템플릿 획들을 참조 $\rightarrow$ 헤드 출력:

출력	형태	의미
overall	(B,)	전체 점수 0~1 (시그모이드 회귀)
q	(B,S)	획별 품질 0~1
rev_logit	(B,S)	필순 방향 반전 확률 (로짓)
ord_logit	(B,S)	획 순서 오류 확률 (로짓)

■■ = $4.0 \cdot \text{MSE}(\text{overall}) + 2.0 \cdot \text{MSE}(q) + 0.5 \cdot (\text{BCE}(\text{rev}) + \text{BCE}(\text{ord}))$, ■■ ■■ ■■■■.

5. 모델 B — Chandra OCR 전이학습

Chandra(datalab-to/chandra)는 Qwen3-VL 기반 8B 오픈소스 OCR 모델로 손글씨 인식이 강점이다. 이 모델의 비전 타워만 떼어내 채점 백본으로 전이학습한다. LLM 부분은 로드 직후 버려 메모리를 절약한다. GPU(VRAM 24GB+) 전용이며 RunPod에서 실행한다.

5.1 시간 인코딩 래스터화 (render.py)

비전 모델은 정지 이미지만 보므로 필순·방향 정보가 사라지는 문제가 있다. 이를 3채널 인코딩으로 해결한다:

채널	내용	모델이 읽을 수 있게 되는 것
ch0 잉크	획이 지나간 자리 (0/1)	글자 모양
ch1 진행도	획 시작 0.15 → 끝 1.0 그라디언트	필기 방향 (어디서 시작해 어디서 끝났나)
ch2 순서	(획번호+1)/총획수 균일값	획을 쓴 순서

동시에 획별 소프트 마스크 (S,448,448)를 만들고, 이를 비전 패치 그리드(16×16=256토큰)로 다운샘플·정규화해 획별 풀링 가중치 (S,256)를 얻는다(masks_to_grid).

5.2 모델 구조 (chandra_scorer.py)

사용자 이미지와 템플릿 이미지를 각각 Chandra 비전 타워(bf16, 기본 동결)에 통과시켜 패치 특징을 얻는다. 사용자 쪽은 획별 풀링 가중치와의 행렬곱으로 획 임베딩 (S,d)을 만들고, 템플릿 쪽은 패치 시퀀스 전체를 메모리로 쓴다. 이후는 모델 A와 동일: 교차어텐션 2층 → q/rev/ord/overall 헤드. 학습 파라미터는 헤드 몇 M뿐이라 빠르다. --unfreeze-last N 으로 백본 마지막 N블록을 낮은 학습률(1e-5)로 함께 미세조정할 수 있다. 체크포인트는 헤드만 저장한다(chandra_scorer.pt) — 백본은 추론 시 HuggingFace에서 다시 받는다.

6. 교정 피드백 엔진 (feedback.py) — "점수 극대화" 제시

analyze()는 다음 순서로 동작한다.

단계	내용
① 전처리	태블릿 원시 좌표 → 12점 재샘플링 + 정규화 (템플릿과 동일 좌표계)
② 획 매칭	탐욕 매칭: 비용 = 1.2·위치오차 + 모양오차 + 0.03· 순서차이 . 대응 없는 템플릿 획 = 누락(missing), 대응 없는 사용자 획 = 추가(extra)
③ 모델 채점	전체 점수·획별 q·방향/순서 확률 예측. 누락 획은 모델이 볼 수 없으므로 점수에 coverage = (전체획수 - 누락)/전체획수 곱함
④ 반사실 분석	각 획을 정답 획으로 교체한 가상 입력을 다시 채점 → 점수 상승분 = "이 획을 고치면 +X점". 상승분 순으로 정렬하면 최상위 3개
⑤ 그래디언트	$\partial(\text{전체점수})/\partial(\text{각 점 좌표})$ 를 역전파로 계산 → 각 점을 어느 방향으로 옮겨야 점수가 오르는지 벡터 필드 (모델 A 전용)
⑥ 메시지 생성	진단 조합 → 한국어 교정 문구: 필순 방향 반대 / 획 순서 오류(N번째로) / 위치 이동(방향 포함) / 모양 교정 / 누락·불일치

리포트 형식(JSON 직렬화 가능):

```
{ score, base_model_score, strokes: [{index, q, rev_prob, ord_prob, pos_err, shape_err, gain, move, messages}], missing, extra, match, grad, corrections }
```

corrections는 기대 점수 상승(gain) 내림차순 — 사용자는 맨 위 획부터 고치면 가장 효율적으로 점수를 올릴 수 있다.

7. 학습 및 서빙 절차

7.1 RunPod 학습

```
bash runpod_train.sh # Chandra █████ (██)
N_CHARS=0 EPOCHS=4 bash runpod_train.sh # ██ ███
UNFREEZE_LAST=4 bash runpod_train.sh # █████ ██████████
MODE=stroke bash runpod_train.sh # █████ ███ A (1████→2████ █████)
```

산출물은 checkpoints/ 에 저장되며, 이를 다운로드해 추론에 쓴다.

7.2 추론 API

```
from scorer.score_api import KanjiScorer # █████ A (CPU █████)
ks = KanjiScorer("checkpoints/scorer.pt", "kanji")
report = ks.score("█", user_strokes)

from scorer.chandra_scorer import load_chandra_scorer, analyze_chandra # █████ B (GPU)
```

7.3 웹 데모 (server.py + static/index.html)

python -m scorer.server --port 8765 로 실행. 브라우저 캔버스가 Pointer Events로 펜/터치/마우스 입력을 받아 획 좌표를 서버로 보내면(POST /score) 채점 리포트가 돌아온다. 획은 품질에 따라 초록/주황/빨강으로 칠해지고, 교정 화살표(현재 위치→정답 위치)와 "+X점" 메시지가 표시된다. 같은 Wi-Fi의 아이패드에서 PC IP로 접속하면 애플펜슬로 쓸 수 있다.

8. 현재 상태와 한계

항목	상태
파이프라인 코드	전체 구현 완료, 로컬 스모크 테스트 통과 (파싱·데이터셋·모델·렌더링)
recognizer.pt	로컬 CPU에서 300자 4에폭 POC 학습 완료 (본격 학습은 RunPod 권장)
scorer.pt / chandra_scorer.pt	미학습 — RunPod에서 runpod_train.sh 실행 필요

Chandra 백본 로드부	GPU가 없어 로컬 미검증 — transformers 버전에 따라 첫 실행 시 조정 가능성
라벨 품질	합성 라벨 기반. 실제 학습자 필기 데이터가 쌓이면 파인튜닝으로 개선 가능